

NETWORK LAB — ASSIGNMENT 2

Computer Networks (PCCST501)

Name	Juvitta Santa Biju
Roll No	43
Class	S5 CSA
Submitted To	Ms. Arsha J K

Experiment 1: Hello — Client/Server Communication (TCP Sockets)

Aim

To implement a basic client–server application using TCP sockets in C, in which the client connects to the server, sends a greeting message (“Hello Server”), and receives a reply (“Hello Client”) from the server, thereby demonstrating a reliable two-way communication channel between two processes over a network.

Software Used

GCC Compiler, Linux Terminal (WSL / Ubuntu)

Commands Used

- gcc server.c -o server
- gcc client.c -o client
- ./server (run in Terminal 1)
- ./client (run in Terminal 2)

Algorithm

Server:

1. Create a TCP socket using `socket()`.
2. Configure the server address structure with `INADDR_ANY` and port 8080.
3. Bind the socket to the address using `bind()`.
4. Put the socket into listening mode using `listen()`.
5. Accept an incoming client connection using `accept()`.
6. Receive the message sent by the client using `recv()`.
7. Display the received message and send the reply “Hello Client” using `send()`.
8. Close the client and server sockets.

Client:

1. Create a TCP socket using `socket()`.
2. Configure the server address structure with IP 127.0.0.1 and port 8080.
3. Connect to the server using `connect()`.
4. Send the message “Hello Server” using `send()`.
5. Receive the server's reply using `recv()` and display it.
6. Close the socket.

Server Code (server.c)

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

int main() {
    int server_socket, client_socket;
    struct sockaddr_in server_address;
    char buffer[1024];

    // Create socket
    server_socket = socket(AF_INET, SOCK_STREAM, 0);
```

```

// Configure server address
server_address.sin_family = AF_INET;
server_address.sin_addr.s_addr = INADDR_ANY;
server_address.sin_port = htons(8080);

// Bind socket to port 8080
bind(server_socket, (struct sockaddr*)&server_address, sizeof(server_address));

// Listen for client
listen(server_socket, 5);
printf("Server is waiting for connection...\n");

// Accept client connection
client_socket = accept(server_socket, NULL, NULL);

// Receive message from client
recv(client_socket, buffer, sizeof(buffer), 0);
printf("Client: %s\n", buffer);

// Send reply
char message[] = "Hello Client";
send(client_socket, message, strlen(message) + 1, 0);

// Close sockets
close(client_socket);
close(server_socket);
return 0;
}

```

Client Code (client.c)

```

#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

int main() {
    int client_socket;
    struct sockaddr_in server_address;
    char buffer[1024];

    // Create socket
    client_socket = socket(AF_INET, SOCK_STREAM, 0);

    // Configure server address
    server_address.sin_family = AF_INET;
    server_address.sin_port = htons(8080);
    server_address.sin_addr.s_addr = inet_addr("127.0.0.1");

    // Connect to server
    connect(client_socket, (struct sockaddr*)&server_address, sizeof(server_address));

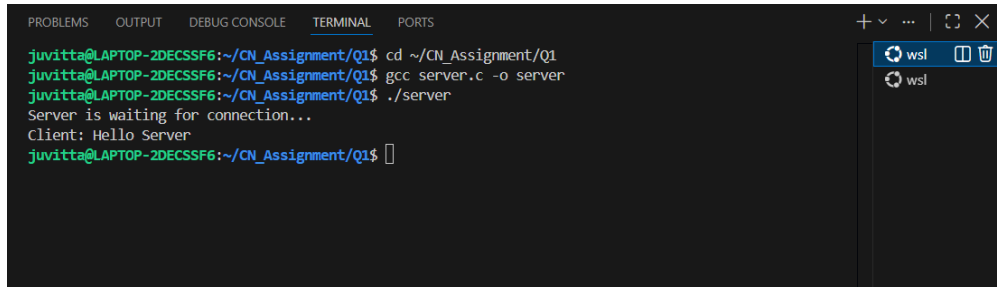
    // Send message
    char message[] = "Hello Server";
    send(client_socket, message, strlen(message) + 1, 0);

    // Receive reply
    recv(client_socket, buffer, sizeof(buffer), 0);
    printf("Server: %s\n", buffer);
}

```

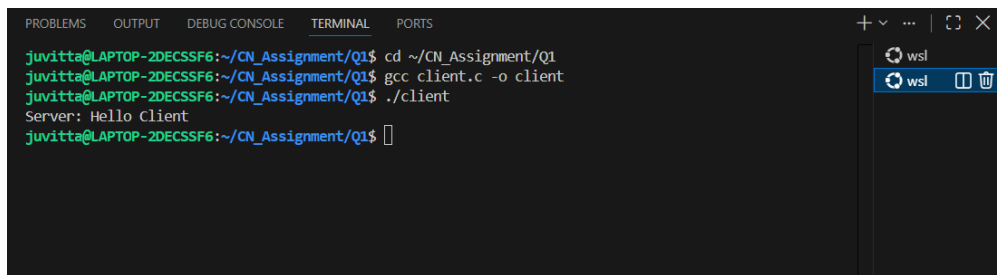
```
// Close socket
close(client_socket);
return 0;
}
```

Output



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q1$ cd ~/CN_Assignment/Q1
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q1$ gcc server.c -o server
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q1$ ./server
Server is waiting for connection...
Client: Hello Server
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q1$
```

Fig 1.1: Server Terminal



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q1$ cd ~/CN_Assignment/Q1
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q1$ gcc client.c -o client
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q1$ ./client
Server: Hello Client
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q1$
```

Fig 1.2: Client Terminal

Conclusion

The client-server program was implemented successfully using TCP sockets. The server was started first and began waiting for a connection on port 8080, after which the client connected to it, sent the message “Hello Server”, and received the reply “Hello Client”. This confirms that a reliable, connection-oriented, two-way communication channel was successfully established between the client and the server processes.

Experiment 2: Arithmetic Server

Aim

To implement a client–server application in which the client sends two integers to the server, and the server computes and returns their sum, difference, product, and quotient, demonstrating exchange of numeric data and remote computation over a TCP socket connection.

Software Used

GCC Compiler, Linux Terminal (WSL / Ubuntu)

Commands Used

- gcc server.c -o server
- gcc client.c -o client
- ./server (run in Terminal 1)
- ./client (run in Terminal 2)

Algorithm

Server:

1. Create a TCP socket and enable SO_REUSEADDR using setsockopt().
2. Bind the socket to INADDR_ANY and port 8081 using bind().
3. Put the socket into listening mode using listen().
4. Accept an incoming client connection using accept().
5. Receive the two integers (sent as a space-separated string) using recv().
6. Parse the integers using sscanf() and compute sum, difference, product, and quotient (checking for division by zero).
7. Format the results into a string and send them back to the client using send().
8. Close the client and server sockets.

Client:

1. Create a TCP socket using socket().
2. Specify the server's IP address (127.0.0.1) and port number (8081).
3. Connect to the server using connect().
4. Read two integers from the user and format them into a message string.
5. Send the message to the server using send().
6. Receive the computed results from the server using recv() and display them.
7. Close the socket.

Server Code (server.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8081
#define BUFFER_SIZE 1024

int main() {
```

```

int server_fd, client_fd;
struct sockaddr_in server_addr, client_addr;
socklen_t client_len = sizeof(client_addr);
char buffer[BUFFER_SIZE] = {0};
char result[BUFFER_SIZE] = {0};

server_fd = socket(AF_INET, SOCK_STREAM, 0);
int opt = 1;
setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

server_addr.sin_family = AF_INET;
server_addr.sin_addr.s_addr = INADDR_ANY;
server_addr.sin_port = htons(PORT);

bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));
listen(server_fd, 3);
printf("Server listening on port %d...\n", PORT);

client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &client_len);
printf("Client connected: %s:%d\n", inet_ntoa(client_addr.sin_addr), ntohs(client_addr.sin_port));

int n = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
buffer[n] = '\0';
printf("Received from client: %s\n", buffer);

int a, b;
sscanf(buffer, "%d %d", &a, &b);
int sum = a + b;
int diff = a - b;
int prod = a * b;

if (b != 0) {
    double quo = (double)a / b;
    snprintf(result, BUFFER_SIZE,
        "Sum = %d\nDifference = %d\nProduct = %d\nQuotient = %.2f",
        sum, diff, prod, quo);
} else {
    snprintf(result, BUFFER_SIZE,
        "Sum = %d\nDifference = %d\nProduct = %d\nQuotient = Undefined (division by zero)",
        sum, diff, prod);
}

send(client_fd, result, strlen(result), 0);
printf("Sent results to client:\n%s\n", result);

close(client_fd);
close(server_fd);
return 0;
}

```

Client Code (client.c)

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8081
#define BUFFER_SIZE 1024
#define SERVER_IP "127.0.0.1"

```

```

int main() {
    int client_fd;
    struct sockaddr_in server_addr;
    char buffer[BUFFER_SIZE] = {0};
    char message[BUFFER_SIZE] = {0};
    int a, b;

    client_fd = socket(AF_INET, SOCK_STREAM, 0);
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, SERVER_IP, &server_addr.sin_addr);

    connect(client_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));
    printf("Connected to server %s:%d\n", SERVER_IP, PORT);

    printf("Enter first integer: ");
    scanf("%d", &a);
    printf("Enter second integer: ");
    scanf("%d", &b);

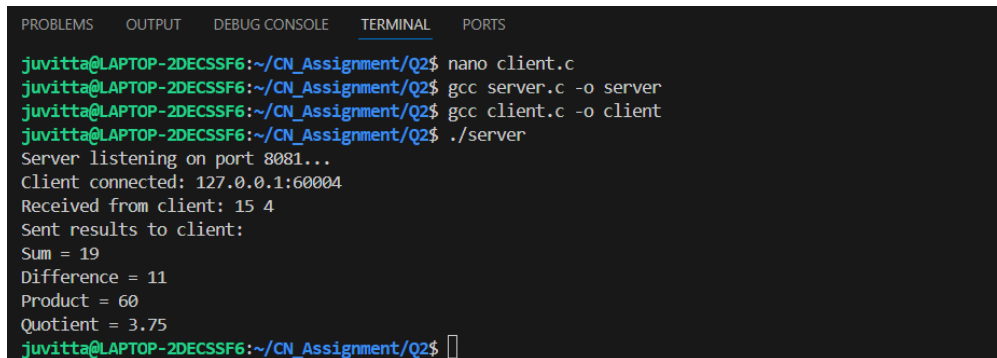
    snprintf(message, BUFFER_SIZE, "%d %d", a, b);
    send(client_fd, message, strlen(message), 0);
    printf("Sent to server: %s\n", message);

    int n = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
    buffer[n] = '\0';
    printf("Results received from server:\n%s\n", buffer);

    close(client_fd);
    return 0;
}

```

Output

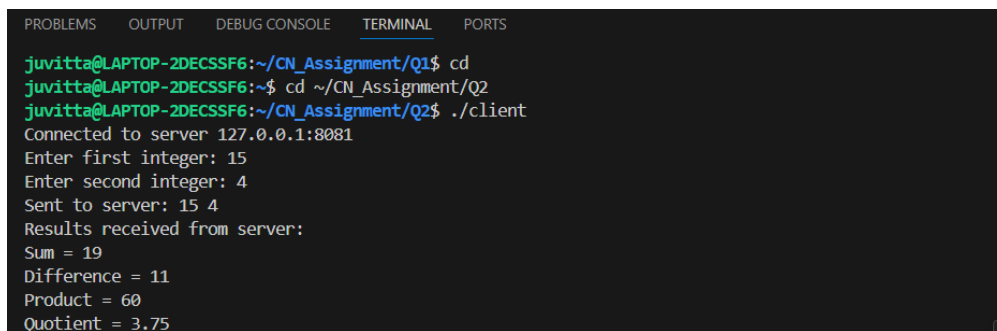


```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q2$ nano client.c
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q2$ gcc server.c -o server
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q2$ gcc client.c -o client
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q2$ ./server
Server listening on port 8081...
Client connected: 127.0.0.1:60004
Received from client: 15 4
Sent results to client:
Sum = 19
Difference = 11
Product = 60
Quotient = 3.75
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q2$

```

Fig 2.1: Server Terminal



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q1$ cd
juvitta@LAPTOP-2DECSSF6:~$ cd ~/CN_Assignment/Q2
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q2$ ./client
Connected to server 127.0.0.1:8081
Enter first integer: 15
Enter second integer: 4
Sent to server: 15 4
Results received from server:
Sum = 19
Difference = 11
Product = 60
Quotient = 3.75

```

Fig 2.2: Client Terminal

Conclusion

The arithmetic server was successfully implemented and tested. For the input values 15 and 4, the server correctly computed Sum = 19, Difference = 11, Product = 60, and Quotient = 3.75, and returned all results to the client over the socket connection, confirming correct numeric data exchange and remote computation.

Experiment 3: String Processing Server

Aim

To implement a client–server application in which the client sends a string to the server, and the server computes the length of the string, converts it to uppercase, and reverses it, then returns all the results to the client, demonstrating string processing over a TCP socket connection.

Software Used

GCC Compiler, Linux Terminal (WSL / Ubuntu)

Commands Used

- gcc server.c -o server
- gcc client.c -o client
- ./server (run in Terminal 1)
- ./client (run in Terminal 2)

Algorithm

Server:

1. Create a TCP socket and enable SO_REUSEADDR using setsockopt().
2. Bind the socket to INADDR_ANY and port 8082 using bind().
3. Put the socket into listening mode using listen().
4. Accept an incoming client connection using accept().
5. Receive the string sent by the client using recv().
6. Compute the length of the string, convert it to uppercase, and reverse it.
7. Format all three results into a string and send them back to the client using send().
8. Close the client and server sockets.

Client:

1. Create a TCP socket using socket().
2. Specify the server's IP address (127.0.0.1) and port number (8082).
3. Connect to the server using connect().
4. Read a string from the user using fgets() and strip the trailing newline.
5. Send the string to the server using send().
6. Receive the processed results from the server using recv() and display them.
7. Close the socket.

Server Code (server.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8082
#define BUFFER_SIZE 1024

int main() {
```

```

int server_fd, client_fd;
struct sockaddr_in server_addr, client_addr;
socklen_t client_len = sizeof(client_addr);
char buffer[BUFFER_SIZE] = {0};
char upper[BUFFER_SIZE] = {0};
char reversed[BUFFER_SIZE] = {0};
char result[BUFFER_SIZE * 3] = {0};

server_fd = socket(AF_INET, SOCK_STREAM, 0);
int opt = 1;
setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

server_addr.sin_family = AF_INET;
server_addr.sin_addr.s_addr = INADDR_ANY;
server_addr.sin_port = htons(PORT);

bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));
listen(server_fd, 3);
printf("Server listening on port %d...\n", PORT);

client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &client_len);
printf("Client connected: %s:%d\n", inet_ntoa(client_addr.sin_addr), ntohs(client_addr.sin_port));

int n = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
buffer[n] = '\0';
printf("Received from client: %s\n", buffer);

int len = strlen(buffer);
for (int i = 0; i < len; i++) upper[i] = toupper(buffer[i]);
upper[len] = '\0';
for (int i = 0; i < len; i++) reversed[i] = buffer[len - 1 - i];
reversed[len] = '\0';

snprintf(result, sizeof(result),
    "Length = %d\nUppercase = %s\nReversed = %s",
    len, upper, reversed);

send(client_fd, result, strlen(result), 0);
printf("Sent results to client:\n%s\n", result);

close(client_fd);
close(server_fd);
return 0;
}

```

Client Code (client.c)

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8082
#define BUFFER_SIZE 1024
#define SERVER_IP "127.0.0.1"

int main() {
    int client_fd;
    struct sockaddr_in server_addr;
    char buffer[BUFFER_SIZE] = {0};

```

```

char input[BUFFER_SIZE];

client_fd = socket(AF_INET, SOCK_STREAM, 0);
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(PORT);
inet_pton(AF_INET, SERVER_IP, &server_addr.sin_addr);

connect(client_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));
printf("Connected to server %s:%d\n", SERVER_IP, PORT);

printf("Enter a string: ");
fgets(input, BUFFER_SIZE, stdin);
input[strcspn(input, "\n")] = '\0';

send(client_fd, input, strlen(input), 0);
printf("Sent to server: %s\n", input);

int n = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
buffer[n] = '\0';
printf("Results received from server:\n%s\n", buffer);

close(client_fd);
return 0;
}

```

Output

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q3$ nano server.c
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q3$ nano client.c
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q3$ gcc server.c -o server
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q3$ gcc client.c -o client
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q3$ ./server
Socket Created Successfully...
Bind Successful...
Waiting for Client...
Client Connected...
Received String: hello world
Results Sent...
Connection Closed...
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q3$ 

```

Fig 3.1: Server Terminal

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
juvitta@LAPTOP-2DECSSF6:~$ cd ~/CN_Assignment/Q3
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q3$ gcc client.c -o client
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q3$ ./client
Socket Created Successfully...
Connected to Server...
Enter a string: hello world

----- Results -----
String Length : 11
Uppercase      : HELLO WORLD
Reverse        : dlrow olleh
Connection Closed...
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q3$ 

```

Fig 3.2: Client Terminal

Conclusion

The string processing server was successfully implemented and tested with the input “hello world”. The server correctly returned String Length = 11, Uppercase = HELLO WORLD, and Reverse = dlrow olleh, confirming successful string manipulation over the socket connection.

Experiment 4: Array Processing Server

Aim

To implement a client–server application in which the client sends an array of N integers to the server, and the server computes the sum, average, maximum, and minimum of the array elements, demonstrating array data transfer and aggregate computation over a TCP socket connection.

Software Used

GCC Compiler, Linux Terminal (WSL / Ubuntu)

Commands Used

- `gcc server.c -o server`
- `gcc client.c -o client`
- `./server` (run in Terminal 1)
- `./client` (run in Terminal 2)

Algorithm

Server:

1. Create a TCP socket and enable `SO_REUSEADDR` using `setsockopt()`.
2. Bind the socket to `INADDR_ANY` and port 8083 using `bind()`.
3. Put the socket into listening mode using `listen()`.
4. Accept an incoming client connection using `accept()`.
5. Receive the message containing N followed by N integers (space-separated) using `recv()`.
6. Tokenize the message using `strtok()` to extract N and the array elements.
7. Compute the sum, average, maximum, and minimum of the array elements.
8. Format the results into a string and send them back to the client using `send()`.
9. Close the client and server sockets.

Client:

1. Create a TCP socket using `socket()`.
2. Specify the server's IP address (127.0.0.1) and port number (8083).
3. Connect to the server using `connect()`.
4. Read the number of elements N and then N integers from the user.
5. Build a single message string containing N followed by the array elements.
6. Send the message to the server using `send()`.
7. Receive the computed results from the server using `recv()` and display them.
8. Close the socket.

Server Code (server.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8083
#define BUFFER_SIZE 1024
```

```

int main() {
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t client_len = sizeof(client_addr);
    char buffer[BUFFER_SIZE] = {0};
    char result[BUFFER_SIZE] = {0};

    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));
    listen(server_fd, 3);
    printf("Server listening on port %d...\n", PORT);

    client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &client_len);
    printf("Client connected: %s:%d\n", inet_ntoa(client_addr.sin_addr), ntohs(client_addr.sin_port));

    int n = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
    buffer[n] = '\0';
    printf("Received from client: %s\n", buffer);

    int arr[100], count = 0;
    char *token = strtok(buffer, " ");
    int N = atoi(token);
    token = strtok(NULL, " ");
    while (token != NULL && count < N) {
        arr[count++] = atoi(token);
        token = strtok(NULL, " ");
    }

    int sum = 0, max = arr[0], min = arr[0];
    for (int i = 0; i < count; i++) {
        sum += arr[i];
        if (arr[i] > max) max = arr[i];
        if (arr[i] < min) min = arr[i];
    }
    double avg = (double)sum / count;

    snprintf(result, BUFFER_SIZE,
        "Sum = %d\nAverage = %.2f\nMaximum = %d\nMinimum = %d",
        sum, avg, max, min);

    send(client_fd, result, strlen(result), 0);
    printf("Sent results to client:\n%s\n", result);

    close(client_fd);
    close(server_fd);
    return 0;
}

```

Client Code (client.c)

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

```

```

#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8083
#define BUFFER_SIZE 1024
#define SERVER_IP "127.0.0.1"

int main() {
    int client_fd;
    struct sockaddr_in server_addr;
    char buffer[BUFFER_SIZE] = {0};
    char message[BUFFER_SIZE] = {0};
    int arr[100], N;

    client_fd = socket(AF_INET, SOCK_STREAM, 0);
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, SERVER_IP, &server_addr.sin_addr);

    connect(client_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));
    printf("Connected to server %s:%d\n", SERVER_IP, PORT);

    printf("Enter number of elements (N): ");
    scanf("%d", &N);
    printf("Enter %d integers:\n", N);

    char temp[16];
    snprintf(message, BUFFER_SIZE, "%d", N);
    for (int i = 0; i < N; i++) {
        scanf("%d", &arr[i]);
        snprintf(temp, sizeof(temp), " %d", arr[i]);
        strcat(message, temp);
    }

    send(client_fd, message, strlen(message), 0);
    printf("Sent to server: %s\n", message);

    int n = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
    buffer[n] = '\0';
    printf("Results received from server:\n%s\n", buffer);

    close(client_fd);
    return 0;
}

```

Output

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q4$ nano client.c
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q4$ gcc server.c -o server
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q4$ gcc client.c -o client
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q4$ ./server
Server listening on port 8083...
Client connected: 127.0.0.1:32932
Received from client: 5 12 7 25 3 18
Sent results to client:
Sum = 65
Average = 13.00
Maximum = 25
Minimum = 3
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q4$

```

Fig 4.1: Server Terminal

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
juvitta@LAPTOP-2DECSSF6:~$ cd ~/CN_Assignment/Q4
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q4$ ./client
Connected to server 127.0.0.1:8083
Enter number of elements (N): 5
Enter 5 integers:
12 7 25 3 18
Sent to server: 5 12 7 25 3 18
Results received from server:
Sum = 65
Average = 13.00
Maximum = 25
Minimum = 3
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q4$
```

Fig 4.2: Client Terminal

Conclusion

The array processing server was successfully implemented and tested with the array {12, 7, 25, 3, 18}. The server correctly computed Sum = 65, Average = 13.00, Maximum = 25, and Minimum = 3, verifying correct array handling and aggregate computation over the client-server model.

Experiment 5: Matrix Analyzer

Aim

To implement a client-server application in which the client sends a square matrix to the server, and the server determines the type of matrix (diagonal, upper triangular, lower triangular, or general), and computes the sum of its elements and its trace, demonstrating two-dimensional array processing over a TCP socket connection.

Software Used

GCC Compiler, Linux Terminal (WSL / Ubuntu)

Commands Used

- gcc server.c -o server
- gcc client.c -o client
- ./server (run in Terminal 1)
- ./client (run in Terminal 2)

Algorithm

Server:

1. Create a TCP socket and enable SO_REUSEADDR using setsockopt().
2. Bind the socket to INADDR_ANY and port 8084 using bind().
3. Put the socket into listening mode using listen().
4. Accept an incoming client connection using accept().
5. Receive the message containing N followed by the N×N matrix elements (space-separated) using recv().
6. Tokenize the message using strtok() to reconstruct N and the matrix.
7. Check the matrix for diagonal, upper triangular, and lower triangular properties, and classify it accordingly.
8. Compute the sum of all elements and the trace (sum of diagonal elements) of the matrix.
9. Format the matrix type, sum, and trace into a string and send them back to the client using send().
10. Close the client and server sockets.

Client:

1. Create a TCP socket using socket().
2. Specify the server's IP address (127.0.0.1) and port number (8084).
3. Connect to the server using connect().
4. Read the order N and then the N×N matrix elements from the user.
5. Build a single message string containing N followed by all the matrix elements.
6. Send the message to the server using send().
7. Receive the computed results (matrix type, sum, trace) from the server using recv() and display them.
8. Close the socket.

Server Code (server.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8084
```



```

#define BUFFER_SIZE 4096
#define MAXN 20

int main() {
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t client_len = sizeof(client_addr);
    char buffer[BUFFER_SIZE] = {0};
    char result[BUFFER_SIZE] = {0};
    int mat[MAXN][MAXN];

    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));
    listen(server_fd, 3);
    printf("Server listening on port %d...\n", PORT);

    client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &client_len);
    printf("Client connected: %s:%d\n", inet_ntoa(client_addr.sin_addr), ntohs(client_addr.sin_port));

    int n = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
    buffer[n] = '\0';
    printf("Received from client: %s\n", buffer);

    char *token = strtok(buffer, " ");
    int N = atoi(token);
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            token = strtok(NULL, " ");
            mat[i][j] = atoi(token);
        }
    }

    int isDiagonal = 1, isUpper = 1, isLower = 1;
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (i != j && mat[i][j] != 0) isDiagonal = 0;
            if (i > j && mat[i][j] != 0) isUpper = 0;
            if (i < j && mat[i][j] != 0) isLower = 0;
        }
    }

    char type[32];
    if (isDiagonal) strcpy(type, "Diagonal Matrix");
    else if (isUpper) strcpy(type, "Upper Triangular Matrix");
    else if (isLower) strcpy(type, "Lower Triangular Matrix");
    else strcpy(type, "General Matrix");

    int sum = 0, trace = 0;
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            sum += mat[i][j];
        }
        trace += mat[i][i];
    }

    snprintf(result, BUFFER_SIZE,

```

```

    "Matrix Type = %s\nSum of Elements = %d\nTrace = %d",
    type, sum, trace);

send(client_fd, result, strlen(result), 0);
printf("Sent results to client:\n%s\n", result);

close(client_fd);
close(server_fd);
return 0;
}

```

Client Code (client.c)

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8084
#define BUFFER_SIZE 4096
#define SERVER_IP "127.0.0.1"
#define MAXN 20

int main() {
    int client_fd;
    struct sockaddr_in server_addr;
    char buffer[BUFFER_SIZE] = {0};
    char message[BUFFER_SIZE] = {0};
    int mat[MAXN][MAXN], N;

    client_fd = socket(AF_INET, SOCK_STREAM, 0);
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, SERVER_IP, &server_addr.sin_addr);

    connect(client_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));
    printf("Connected to server %s:%d\n", SERVER_IP, PORT);

    printf("Enter order of square matrix (N): ");
    scanf("%d", &N);
    printf("Enter %d x %d matrix elements:\n", N, N);

    char temp[16];
    snprintf(message, BUFFER_SIZE, "%d", N);
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            scanf("%d", &mat[i][j]);
            snprintf(temp, sizeof(temp), " %d", mat[i][j]);
            strcat(message, temp);
        }
    }

    send(client_fd, message, strlen(message), 0);
    printf("Sent to server: %s\n", message);

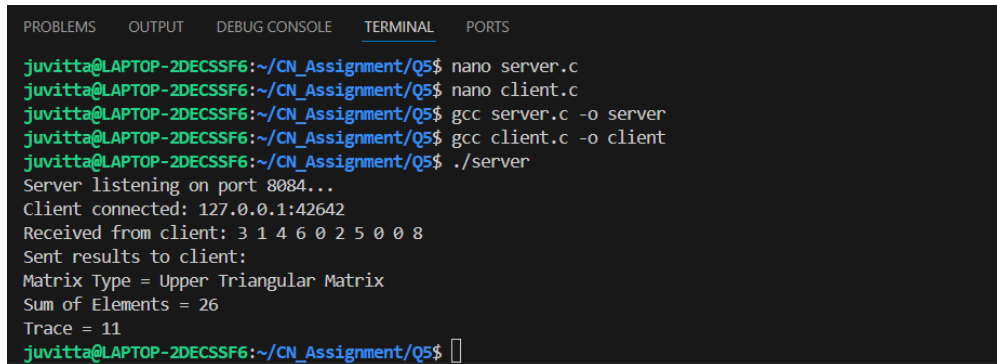
    int n = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
    buffer[n] = '\0';
    printf("Results received from server:\n%s\n", buffer);

    close(client_fd);
}

```

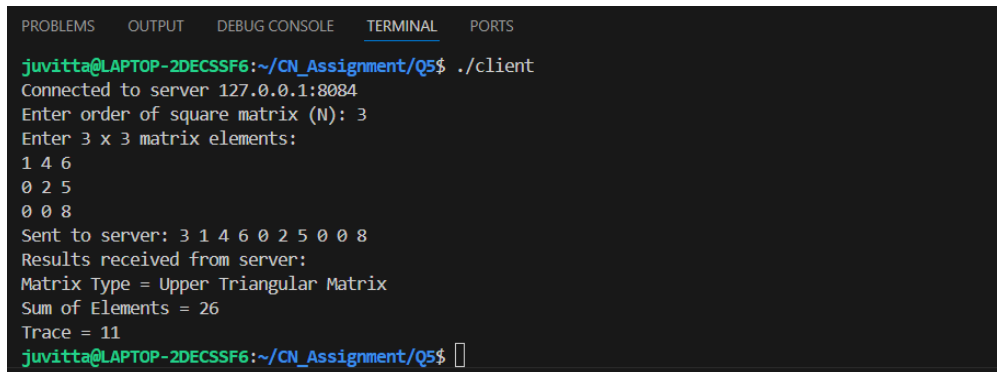
```
    return 0;
}
```

Output



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q5$ nano server.c
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q5$ nano client.c
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q5$ gcc server.c -o server
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q5$ gcc client.c -o client
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q5$ ./server
Server listening on port 8084...
Client connected: 127.0.0.1:42642
Received from client: 3 1 4 6 0 2 5 0 8
Sent results to client:
Matrix Type = Upper Triangular Matrix
Sum of Elements = 26
Trace = 11
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q5$
```

Fig 5.1: Server Terminal



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q5$ ./client
Connected to server 127.0.0.1:8084
Enter order of square matrix (N): 3
Enter 3 x 3 matrix elements:
1 4 6
0 2 5
0 0 8
Sent to server: 3 1 4 6 0 2 5 0 8
Results received from server:
Matrix Type = Upper Triangular Matrix
Sum of Elements = 26
Trace = 11
juvitta@LAPTOP-2DECSSF6:~/CN_Assignment/Q5$
```

Fig 5.2: Client Terminal

Conclusion

The matrix analyzer server was successfully implemented and tested with a 3×3 upper triangular matrix. The server correctly identified the matrix type as “Upper Triangular Matrix”, and computed Sum of Elements = 26 and Trace = 11, confirming correct matrix analysis over the socket connection.